Chapitre 4 : Programmation orientée objet (POO)

IN323 - Programmation en langage C++

Elisa HENRION-GUENEAU

IPSA - Site Toulouse

AERO3 - 2025/2026

Conception d'une Classe

Introduction POO

Concepts et principes de base :

- **Classe** : Modèle ou plan pour créer des objets
- **Objet** : Instance d'une classe
- **Encapsulation** : Regrouper des données et les fonctions qui les manipulent (protéger les données avec `private`, `protected`, `public`)
- **Héritage** : Permet à une classe d'hériter des propriétés d'une autre classe.

Avantages :

- Code réutilisable
- Modélisation plus simple pour des systèmes complexes
- Meilleure organisation et maintenance du code

Exemple simple : La classe Point

```
class Point {
public:
    // attributs publics
    int x, y;

    // Constructeur : initialise les coordonnees
    Point(int valX, int valY) {
        x = valX;
        y = valY;
    }
    // Methode (virtuelle, on verra plus tard)
    virtual void afficher() {
        cout <<"("<< x <<","<< y <<")"<< endl;
    }
};
```

Instanciation simple

L'objet est créé et initialisé en une seule ligne.

```
int main() {
    // Appelle le constructeur Point(int, int)
    Point p1(10, 20);

    p1.afficher(); // Affiche (10,20)

    p1.x = 15; // possible car x est public
    return 0;
}
```

Les attributs

Définition :

- Représentent les caractéristiques/propriétés d'un objet
- Définissent l'état d'un objet à un instant donné.

Caractéristiques :

- **Nom :** nom unique pour chaque attribut
- **Type :** entier, string, booléen, etc...
- **Valeur :** valeurs spécifiques pour chaque objet

Cas particulier : attribut **static**, commun à toutes les instances

Retour sur la classe Point

```
class Point {
public:
    int x, y;
    static int nbPoints; // attribut commun
    Point(int valX, int valY) {
        x = valX;
        y = valY;
        nbPoints++; // incremente a chaque creation
    }
};
// INITIALISATION OBLIGATOIRE (hors de la classe)
int Point::nbPoints = 0;
int main() {
    Point p1(10,20);
    Point p2(5,5);
    cout <<"nb de points: "<<Point::nbPoints<<endl;
    return 0;
}
```

Visibilité

En laissant les attributs en `public`, n'importe qui peut modifier les données de l'objet de manière incohérente.

Visibilité : détermine qui a accès aux attributs/méthodes.

```
class Exemple {
    private :
        // accessible uniquement dans cette classe.
        int secret;
    protected :
        // dans cette classe et ses sous-classes.
        int protege;
    public :
        // depuis n'importe ou
        int ouvert;
};
```

Les méthodes

Définition :

- Fonction associée à une classe.
- Elle a accès par défaut à tous les attributs de la classe.

Méthodes d'instance :

- Agissent sur les attributs d'un objet particulier.
- Nécessitent une instance pour être appelées.

Méthodes statiques :

- Associées à la classe plutôt qu'à une instance spécifique.
- Ne peuvent accéder qu'aux attributs statiques.

Exemple

```
class Point {
private:
    int x, y;
    static int nbPoints; // attribut commun
public:
    Point(int valX, int valY) {
        x = valX; y = valY;
        nbPoints++;
    }
    static int getNbPoints(){ // Methode statique
        return nbPoints;
    }
};
int Point::nbPoints = 0;
int main() {
    cout <<"nb de points: "<<Point::getNbPoints()<<
        endl;
}
```

Cycle de vie et mémoire

Les constructeurs

Définition :

- Méthode spéciale pour initialiser un objet lors de sa création.
- Nom identique à celui de la classe.
- Pas de type de retour (même pas `void`).

Caractéristiques :

- **Initialisation des Attributs** : attribuer des valeurs initiales aux attributs d'un objet.
- **Surcharge possible** : plusieurs constructeurs avec des paramètres différents.
- **Appelé automatiquement** : constructeur invoqué lors de la création de l'objet.

Mot clé `this` : pointeur vers l'objet courant (pour éviter les ambiguïtés de nom de variables)

Exemple

```
class Point {
private:
    int x, y;
public:
    Point() { // Constructeur par defaut
        this->x = 0;
        this->y = 0;
    }
    Point(int x, int y) { // avec parametres
        this->x = x; // 'this->x' : attribut
        this->y = y; // 'y' : parametre
    }
};
int main() {
    Point p1; // constructeur par defaut (0,0)
    Point p2(5, 12); // second constructeur (5,12)
    return 0;
}
```

Les destructeurs

Définition :

- Méthode spéciale utilisée pour libérer les ressources d'un objet avant sa destruction.
- Porte le même nom que la classe, précédé d'un $\sim$

Caractéristiques :

- **Unique**
- **Sans paramètre** : ne peut pas être surchargé
- **Automatique** : appelé automatiquement à la fin du cycle de vie d'un objet

Exemple

```
class Point {
private:
    int x, y;
public:
    Point() {
        cout << "Creation d'un point" << endl;
    }
    ~Point() {
        cout <<"Destruction de ("<<x<<","<<y<<")"<<
            endl;
    }
};
int main() {
    Point p1 (1,2); // constructeur appele
    ...
    // p1 sera detruit a la fin du main
    return 0;
}
```

Constructeurs/Destructeurs

- Toujours faire un constructeur par défaut
- Faire un constructeur différent par possibilité de construction de l'objet
- Le destructeur sert à libérer la mémoire
- Le destructeur est appelé automatiquement lors de l'utilisation de `delete`

Allocation mémoire

Allocation statique :

- L'objet est détruit automatiquement à la fin de sa portée.

Allocation dynamique :

- Créer avec `new`
- Permet une gestion fine de la durée de vie des objets.
- Doit être libérée manuellement pour éviter les fuites de mémoire avec `delete`

Pour utiliser une méthode ou avoir accès à un attribut à partir d'un pointeur sur un objet il faut utiliser `->` à la place de `.`

Exemple : Statique vs Dynamique

```
int main() {
    // Allocation statique
    Point p1(10, 20);
    p1.afficher(); // Utilisation du '.'

    // Allocation dynamique
    Point* ptr = new Point(5, 5);
    ptr->afficher(); // Utilisation de '->'
    // ou
    (*ptr).afficher();
    ptr->x = 15; // Modification (si autorisee)
    delete ptr; // LIBERATION OBLIGATOIRE

    return 0; // p1 est detruit automatiquement ici
}
```

Héritage et polymorphisme

Le concept d'héritage

Définition :

- Permet de créer une nouvelle classe (fille/dérivée) en se basant sur une classe existante (mère/de base).
- La classe fille "hérite" des attributs et des méthodes de la classe mère.

Avantages :

- **Réutilisation du code** : ne pas réécrire ce qui existe déjà.
- **Spécialisation** : la classe fille ajoute des fonctionnalités spécifiques.
- **Hiérarchie** : organiser les objets du plus général au plus précis.

Exemple

```
class Point {
protected: // Accessible aux enfants
    int x, y;
public:
    ...
};
class Point3D : public Point { // herite de Point
private:
    int z;
public:
    Point3D(int x, int y, int z) : Point(x,y) {
        this->z = z;
    }
    // Redefinition de la methode afficher
    virtual void afficher() override {
        cout <<"("<< x <<","<< y <<","<<z<<")"<<
            endl;
    }
```

Redéfinition

Définition :

- Consiste à remplacer une méthode de la classe mère dans une classe fille.
- Permet d'adapter le comportement d'une méthode héritée selon les besoins spécifiques de la classe fille

Avantages :

- **Même signature** : La méthode redéfinie doit avoir le même nom et les mêmes paramètres.
- Mot clé `override`

Le problème de la liaison statique

Que se passe-t-il si on manipule un `Point3D` via un pointeur de la classe mère ?

```
int main() {
    // Un pointeur de type "Point" qui pointe sur
       un "Point3D"
    Point* p = new Point3D(1, 2, 3);
    p->afficher(); // Appelle Point::afficher()
    delete p;
    return 0;
}
```

Par défaut, le compilateur choisit la méthode selon le **type du pointeur** (`Point`) et non l'objet réel. Pour corriger cela, il faut rendre la méthode **virtuelle**.

Le Polymorphisme (virtual)

En ajoutant `virtual`, le programme choisit la méthode selon le **type réel** de l'objet à l'exécution.

```
class Point {
public:
    // virtual : autorise la redefinition dynamique
    virtual void afficher() {
        cout << "(" << x << "," << y << ")";
    }

    // Toujours mettre le destructeur virtuel
    virtual ~Point() {}
};
```

Classe abstraite

Définition :

- Classe qui ne peut pas être instanciée directement.
- Sert de modèle pour des classes filles.
- Une classe est abstraite si elle contient au moins une méthode virtuelle pure.

Méthode Virtuelle Pure :

- Déclarée avec = 0 dans la classe mère.
- Force les classes filles à fournir leur propre implémentation

Exemple : Forme et Point

```
class Forme {
protected:
    Point centre; // classe Point
public:
    Forme(int x, int y) : centre(x, y) {}
    virtual double calculerAire() = 0;
    virtual ~Forme() {}
};
class Cercle : public Forme {
    int rayon;
public:
    Cercle(int x, int y, int r) : Forme(x, y),
        rayon(r) {}
    double calculerAire() override { //
        Implementation obligatoire
         return 3.14 * rayon * rayon;
    }
};
```